

Урок 9. Корутины. Билдеры launch и async.

11 февраля 2020

В этом уроке рассмотрим билдеры launch и async. Как ведут себя вложенные корутины. Как дождаться окончания работы корутины, и как получить результат этой работы. Как создать отложенную корутину и запустить ее позже. Как распараллеливать задачи с помощью корутин.

Продолжаем запускать корутины и разбираться в их поведении.

Создайте проект аналогично тому, как мы создавали в уроке 8. Кнопки назовите Run и Run2. По нажатию будем вызывать методы onRun и onRun2.

launch

Начнем с билдера launch. Мы уже использовали его в прошлых уроках, чтобы создавать корутины. Но у него есть еще пара возможностей, о которых надо рассказать отдельно. Да и про вложенные корутины уже можно начинать говорить.

join

В прошлом уроке мы рассматривали такой пример:

```
1 private fun onRun() {  
2     scope.launch {  
3         // ...  
4     }  
5  
6     // ...  
7 }
```

Мы выяснили, что билдер launch создает и запускает корутину. Билдер не ждет, пока корутина выполнится, и не блокирует выполнение метода onRun. Билдер быстро делает свое дело, метод onRun продолжает выполнять свой дальнейший код и запросто может завершиться раньше, чем корутина из него запущенная. Это стандартное поведение мультипоточного кода.

Но иногда может возникнуть необходимость дождаться выполнения корутины. Для этого используется метод Job.join()

Схематично это выглядит так:

```
1 job = scope.launch {  
2     // ...  
3 }  
4  
5 // ...  
6  
7 job.join()
```

Билдер launch создает и запускает корутину и возвращает ее job. Используя метод join мы приостанавливаем выполнение кода, пока корутина не завершит работу. Т.е. мы запускаем работу в новом потоке, а в своем потоке ждем, пока работа не завершится.

Но тут есть нюанс. Обратите внимание, я сказал, что join приостанавливает, а не блокирует выполнение кода. Это suspend функция. А значит ее надо вызывать внутри корутины.

Вызовем корутину внутри корутины:

```
1 scope.launch {  
2  
3     job = launch {  
4         // ...  
5     }  
6  
7     // ...  
8  
9     job.join()  
10  
11 }
```

Вызываем launch внутри launch, тем самым создавая дочернюю корутину внутри родительской. В родительской корутине мы можем вызвать suspend функцию join, чтобы дождаться завершения работы дочерней корутины.

Перед тем, как посмотреть это на примере, давайте сначала проверим, как в целом работают вложенные корутины. Пока что без join.

```

1 private fun onRun() {
2     scope.launch {
3         log("parent coroutine, start")
4
5         launch {
6             log("child coroutine, start")
7             TimeUnit.MILLISECONDS.sleep(1000)
8             log("child coroutine, end")
9         }
10
11         log("parent coroutine, end")
12     }
13 }

```

В методе onRun стартуем корутину, а внутри нее еще одну корутину. Если вас смущает, что для создания дочерней корутины не используется явный scope, то посмотрите Урок 7. В нем я подробно разбираю откуда берется scope для дочерней корутины.

В метод onRun я логи не ставлю, потому что мы уже знаем, как в обычном методе ведет себя билдер. Он запускает корутину и никак не задерживает метод onRun.

Нам сейчас интересно, как ведет себя билдер launch не в обычном методе, а внутри корутины. А ведет он себя точно также. Билдер, вызванный в корутине не задерживает выполнение этой корутины. Он только создает и запускает дочернюю корутину, которая уходит работать в свой поток. А родительская корутина продолжает работу. Т.е. поведение полностью аналогично тому, что мы ранее рассматривали.

Смотрим логи:

```

20:32:16.221 parent coroutine, start [DefaultDispatcher-worker-1]
20:32:16.222 parent coroutine, end [DefaultDispatcher-worker-1]
20:32:16.222 child coroutine, start [DefaultDispatcher-worker-3]
20:32:17.224 child coroutine, end [DefaultDispatcher-worker-3]

```

Родительская корутина выполнила весь свой код не дожидаясь выполнения дочерней. А дочерняя корутина в отдельном потоке выполнила свой код.

Хоть родительская корутина и выполнила сразу же весь свой код, но ее статус поменяется на Завершена только когда выполнится дочерняя корутина. Потому что родительская корутина подписывается на дочерние и ждет их завершения, прежде чем официально завершиться. Т.е. то, что корутина выполнила свой код, может и не означать, что она имеет статус Завершена. Об этом еще поговорим в отдельном уроке.

Метод join дает нам возможность поменять ход событий и попросить код родительской корутины подождать, пока не выполнится дочерняя корутина.

Добавляем вызов join в родительскую корутину:

```

1 private fun onRun() {
2     scope.launch {
3         log("parent coroutine, start")
4
5         val job = launch {
6             log("child coroutine, start")
7             TimeUnit.MILLISECONDS.sleep(1000)
8             log("child coroutine, end")
9         }
10
11         log("parent coroutine, wait until child completes")
12         job.join()
13
14         log("parent coroutine, end")
15     }
16 }

```

Теперь в точке вызова join родительская корутина будет ждать, пока не выполнится дочерняя. Напомню, что join - это suspend функция, поэтому она только приостановит выполнение родительской корутины, но не заблокирует ее поток.

Возможно, этот пример кажется вам бессмысленным. Зачем запускать дочернюю корутину в новом фоновом потоке, если родительская корутина и так уже выполняется в фоновом потоке. Можно было бы всю работу сделать в родительской корутине и не плодить потоки.

И вот тут я вам должен сказать очень интересную вещь, которая, может поменять ваше текущее представление о корутинах. Постарайтесь понять следующее утверждение, т.к. это очень важно:

Корутина может выполняться в main потоке.

Выше рассмотренный пример станет выглядеть гораздо более осмысленным и полезным, если код родительской корутины выполняется в main потоке, а код дочерней - в фоновом. Получается, что из main потока мы запустили фоновую работу и подождали ее окончания без каких-либо блокировок и колбэков. join не заблокирует main поток, а только приостановит выполнение кода. Подробнее об этом механизме читайте в первых уроках этого курса.

Подобный пример с main потоком мы еще сделаем в уроке про диспетчеры. И там все станет окончательно понятно. Пока же продолжаем работать в фоновых потоках.

Если возникает вопрос, почему то же самое нельзя без родительской корутины проверить, то вспомните, что join - это suspend функция. Она не может быть вызвана вне корутины.

Смотрим логи:

```

20:40:01.927 parent coroutine, start [DefaultDispatcher-worker-1]
20:40:01.928 parent coroutine, wait for child [DefaultDispatcher-worker-1]
20:40:01.928 child coroutine, start [DefaultDispatcher-worker-2]
20:40:02.930 child coroutine, end [DefaultDispatcher-worker-2]
20:40:02.931 parent coroutine, end [DefaultDispatcher-worker-2]

```

Видим, что код родительской корутины завершился после того, как отработала дочерняя корутина.

Обратите внимание на потоки. Родительская корутина начала свою работу в потоке `DefaultDispatcher-worker-1`, а завершила в потоке `DefaultDispatcher-worker-2`, в котором дочерняя корутина выполнялась. Это особенность работы `suspend` функций и диспетчеров. Позже мы еще вернемся к этому моменту. Но сразу скажу, что, если бы родительская корутина выполнялась в `main` потоке, то такого бы не произошло.

Параллельная работа

Таким образом мы можем распараллеливать несколько работ и дожидаться, пока все они завершатся. Рассмотрим на простом примере с двумя дочерними корутинами:

```
1 private fun onRun() {
2     scope.launch {
3         log("parent coroutine, start")
4
5         val job = launch {
6             TimeUnit.MILLISECONDS.sleep(1000)
7         }
8
9         val job2 = launch {
10            TimeUnit.MILLISECONDS.sleep(1500)
11        }
12
13        log("parent coroutine, wait until children complete")
14        job.join()
15        job2.join()
16
17        log("parent coroutine, end")
18    }
19 }
```

Запускаем дочерние корутины, а потом для обеих вызываем `join`, тем самым дожидаясь окончания их работы. Дочерние корутины отработают параллельно, поэтому общее время работы родительской корутины составит 1500.

Логи:

```
20:46:38.889 parent coroutine, start [DefaultDispatcher-worker-1]
20:46:38.893 parent coroutine, wait until children complete [DefaultDispatcher-worker-1]
20:46:40.395 parent coroutine, end [DefaultDispatcher-worker-3]
```

Lazy

Иногда бывает необходимость стартовать корутину не сразу после создания, а позже. Это может быть полезным, когда вы только создаете корутину, а ее `job` передаете в другой метод, чтобы там уже принять решение - стартовать корутину или нет.

В нашем примере можно сделать так, чтобы по нажатию на кнопку `Run` корутина только создавалась. А работать начинала по нажатию на кнопку `Run2`.

```
1 lateinit var job: Job
2
3 private fun onRun() {
4     log("onRun, start")
5
6     job = scope.launch(start = CoroutineStart.LAZY) {
7         log("coroutine, start")
8         TimeUnit.MILLISECONDS.sleep(1000)
9         log("coroutine, end")
10    }
11
12    log("onRun, end")
13 }
14
15 private fun onRun2() {
16     log("onRun2, start")
17     job.start()
18     log("onRun2, end")
19 }
```

В методе `onRun` создаем корутину с параметром `start = LAZY`. Это не даст корутине начать работу сразу после создания. Полученный при этом `job` используем в методе `onRun2`, чтобы вызвать метод `start`, который и приведет к выполнению корутины.

Логи:

```
11:28:30.526 onRun, start [main]
11:28:30.532 onRun, end [main]
11:28:32.317 onRun2, start [main]
11:28:32.321 onRun2, end [main]
11:28:32.328 coroutine, start [DefaultDispatcher-worker-1]
11:28:33.331 coroutine, end [DefaultDispatcher-worker-1]
```

Жмем кнопку `Run`. Метод `onRun` создает корутину и завершается. Но корутина не стартует. Через пару секунд жмем кнопку `Run2`, и корутина начинает работу. Метод `start` ничего не блокирует. Он стартует корутину в отдельном потоке, а метод `onRun2` быстро завершается.

async

Билдер `async` похож на `launch`. Он также создает и стартует корутину. Но если `launch` корутина делала свою работу и ничего не возвращала в ответ, то `async` корутина может вернуть результат своей работы.

Как вы понимаете, чтобы получить результат работы корутины, надо дождаться окончания ее работы. Это очень похоже на ранее рассмотренный нами метод `join`. В случае с `async` это будет метод `await`. Он не только дожждется окончания работы корутины, но и вернет результат ее работы.

Схематично это выглядит так:

```
1  val deferred = scope.async() {
2      // ...
3      "async result"
4  }
5
6  // ...
7
8  val result = deferred.await()
```

Вместо `Job` мы получаем `Deferred`. Это наследник `Job`, поэтому имеет все те же методы. Но дополнительно у него есть методы для получения результата работы корутины. Один из них - метод `await`.

Как и `join`, этот метод является `suspend` функцией, а значит просто так в обычном методе его не запустить. Нужна корутина.

Рассмотрим пример, аналогичный ранее рассмотренному примеру с `launch`. Только вместо `launch+join` используем `async+await`.

```
1  private fun onRun() {
2      scope.launch {
3          log("parent coroutine, start")
4
5          val deferred = async() {
6              log("child coroutine, start")
7              TimeUnit.MILLISECONDS.sleep(1000)
8              log("child coroutine, end")
9              "async result"
10         }
11
12         log("parent coroutine, wait until child returns result")
13         val result = deferred.await()
14         log("parent coroutine, child returns: $result")
15
16         log("parent coroutine, end")
17     }
18 }
```

В родительской корутине билдером `async` создаем и стартуем дочернюю корутину. Билдер возвращает нам `Deferred` объект. Его метод `await` приостановит выполнение родительской корутины, пока не выполнится дочерняя, и вернет результат дочерней корутины.

Дочерняя корутина имитирует работу в течение одной секунды, а затем возвращает строку `"async result"`.

Логи:

```
parent coroutine, start [DefaultDispatcher-worker-1]
parent coroutine, wait until child returns result [DefaultDispatcher-worker-1]
child coroutine, start [DefaultDispatcher-worker-2]
child coroutine, end [DefaultDispatcher-worker-2]
parent coroutine, child returns: async result [DefaultDispatcher-worker-2]
parent coroutine, end [DefaultDispatcher-worker-2]
```

Все аналогично примеру с `launch+join`. Билдер `async` создает и запускает дочернюю корутину. Родительская корутина продолжает выполняться и останавливается на методе `await`. Теперь она в ожидании завершения дочерней корутины.

Дочерняя корутина выполняется в отдельном потоке. По ее завершению метод `await` возвращает результат ее работы (строка `"async result"`), и возобновляет выполнение кода родительской корутины.

Параллельная работа

С помощью `async` можно распараллеливать работу `suspend` функций.

Рассмотрим простой пример с двумя функциями. Запустим их сначала без `async`:

```

1 private fun onRun() {
2     scope.launch {
3         log("parent coroutine, start")
4
5         val data = getData()
6         val data2 = getData2()
7         val result = "${data}, ${data2}"
8         log("parent coroutine, children returned: $result")
9
10        log("parent coroutine, end")
11    }
12 }
13
14 private suspend fun getData(): String {
15     delay(1000)
16     return "data"
17 }
18
19 private suspend fun getData2(): String {
20     delay(1500)
21     return "data2"
22 }

```

В корутине запускаем две suspend функции: `getData` и `getData2`. Первая имитирует выполнение работы в течение 1000 мс и возвращает строку "data", а вторая - в течение 1500 мс и возвращает строку "data2".

Логи

```

11:06:35.026 parent coroutine, start [DefaultDispatcher-worker-1]
11:06:37.543 parent coroutine, children returned: data, data2 [DefaultDispatcher-worker-1]
11:06:37.546 parent coroutine, end [DefaultDispatcher-worker-1]

```

По времени в логах видно, что общее время выполнения корутины составило около 2500 мсек. Это верно, т.к. suspend функции были вызваны последовательно одна за другой.

С помощью `async` мы можем вызвать эти функции параллельно, чтобы сократить общее время их выполнения. Помещаем вызов каждой suspend функции в отдельную `async` корутину. И методом `await` получаем результат.

```

1 private fun onRun() {
2     scope.launch {
3         log("parent coroutine, start")
4
5         val data = async { getData() }
6         val data2 = async { getData2() }
7
8         log("parent coroutine, wait until children return result")
9         val result = "${data.await()}, ${data2.await()}"
10        log("parent coroutine, children returned: $result")
11
12        log("parent coroutine, end")
13    }
14 }
15
16 private suspend fun getData(): String {
17     delay(1000)
18     return "data"
19 }
20
21 private suspend fun getData2(): String {
22     delay(1500)
23     return "data2"
24 }

```

Логи:

```

11:00:52.751 parent coroutine, start [DefaultDispatcher-worker-1]
11:00:52.758 parent coroutine, wait until children return result [DefaultDispatcher-worker-1]
11:00:54.268 parent coroutine, children returned: data, data2 [DefaultDispatcher-worker-3]
11:00:54.269 parent coroutine, end [DefaultDispatcher-worker-3]

```

Теперь логи показывают, что общее время выполнения родительской корутины составило около 1500 мсек. Наши suspend функции выполнялись параллельно в разных `async` корутинах, а родительская корутину дождалась выполнения обеих и получила результаты выполнения.

А теперь снова представьте, что родительская корутину выполняется в `main` потоке. А suspend-функции - это получение данных с сервера с помощью, например, `Retrofit`. Вы выполняете запросы в фоновых потоках, получаете данные в `main` потоке и отображаете их не в логе, а на экране. И никаких колбэков. В этом и есть одна из основных возможностей корутин - лаконичный асинхронный код.

Lazy

`async` корутину также можно запускать в режиме `Lazy`. Метод `await` стартует ее выполнение.

Закрепление темы

Рекомендую вам поработать с выше рассмотренными примерами, чтобы закрепить понимание темы. Можно добавлять корутины, менять тайминги, экспериментировать с отложенным стартом. А в одном из ближайших уроков мы добавим и работу с потоками.

В качестве проверки того, что материал усвоен, и, что вы теперь точно понимаете, как ведут себя `launch` и `asunc` корутины, вы можете проделать одно упражнение. Посмотрев на любой пример из этого урока, запишите, какие логи должны получиться после его работы, и сверьте результаты с логами из урока. На потоки пока особо не смотрите. Важно понимать порядок выполнения и время.